目录

OpenSAI 下一版本开发计划 **1**
0. 背景与目标 . . . 1
里程碑依赖关系 . . . 2
当前已具备的基础（本版可直接复用，无需重做） . . . 2
M1 • OpenClaw 插件 MVP（最高优先，跑在现有栈上） . . . 2
任务分解 . . . 2
交付物 . . . 3
依赖 . . . 3
风险与应对 . . . 3
验收标准 . . . 4
M2 • Tailscale 跨网组网 . . . 4
任务分解 . . . 4
交付物 . . . 4
依赖 . . . 4
风险与应对 . . . 5
验收标准 . . . 5
M3 • 跨平台设备端 . . . 5
任务分解 . . . 5
交付物 . . . 6
依赖 . . . 6
风险与应对 . . . 6
验收标准 . . . 6
横切关注点 . . . 6
架构改动总览 . . . 6

OpenSAI 下一版本开发计划

本计划聚焦**技术内容**（任务分解、交付物、依赖、风险、验收标准）。**排期与人力由负责人另行补充，本文不涉及。**

0. 背景与目标

把 OpenSAI 从“局域网内测试平台”演进为**可被 Agent 调用、可跨网络组网、可跨平台运行**的分布式算力底座。

三大需求 → 三条工作流，**主线：插件 → 组网 → 平台**：- **M1 插件化（最高优先）**：把 OpenSAI 做成 OpenClaw 的原生插件（AgentSkill），让 Agent 把**预置算法**编排成 DAG、卸载到设备群、取回结果。**仅依赖现有桌面栈即可先行落地。** - **M2 跨网组网**：各端安装官方 Tailscale 客户端，让控制面与数据面都跑在 tailnet 上，使 M1 的插件能把任务分发到**真正跨网**的设备。- **M3 跨平台**：Win/Linux/macOS 作一等计算节点，Android 作**瘦客户端**（采集/轻任务/展示，不跑重模型）。

关键约束（已确认）：- Agent 边界 = **预置算法库**。- 组网 = **各端安装 Tailscale 客户端**。- Android = **瘦客**

户端。- 插件形态 = **原生 OpenClaw AgentSkill（Python，放 `skills/`）**。

里程碑依赖关系

- **M1 独立**：跑在现有 Windows/桌面或局域网栈上，不依赖 M2/M3。
- **M2 依赖 M1**：用插件来演示“跨网卸载”的价值。
- **M3 依赖 M2**：Android 瘦客户端经 tailnet 接入；混合 OS 设备群跨网协作才有意义（桌面跨平台也可先在局域网验证）。
- 插件（M1）的价值随 M2/M3 落地而递增（设备池从单机 → 跨网 → 异构多端）。

当前已具备的基础（本版可直接复用，无需重做）

本版不是从零开始——上一阶段已落地并经回归/断言验证的能力，构成 M1/M2/M3 的现成底座：- **智能放置（已链路感知）**：大脑按各设备**实测链路带宽**（`observed_links`）估算传输代价来选设备，已具备就近聚合 / 容量感知 / 健康预筛。- **多链路数据面（已实现）**：设备间产物传输支持**中继 + 多网卡并行聚合**——天然适配跨网带宽瓶颈与 DERP 中继回退。- **四维资源控制（已实现）**：每个子任务按声明 / 历史自动定档 CPU 核 / 内存 / 带宽 / 存储，账本防超卖。- **聚合监控视图 + 决策事件流 + Web 桥接接口（已实现）**：设备 / 算法 / 资源 / 链路 / 放置 • 资源 • 传输决策全部可读，已有 `/api/devices`、`/api/algorithms`、`/api/events`、`/api/dag/upload` 等 HTTP 端点。- **前端控制台（已实现）**：集群三维可视化、决策过程、拓扑编辑的现成 UI。- **结构化回归（已具备）**：功能测试计划 + 30 条综合功能断言 + 多链路 / 事件 / 桥接断言，作为后续各里程碑“并入回归”的基座。

因此下方多处任务从“新建”降为“在已有基础上封装 / 扩展”，已在对应条目标注 [**已具备基础**]。

M1 ·OpenClaw 插件 MVP（最高优先，跑在现有栈上）

目标：以原生 OpenClaw AgentSkill 形式暴露 OpenSAI 能力，让 Agent 用**已注册算法**编排 DAG、卸载执行、取回结果；在**现有 Windows/桌面单机或局域网栈**上端到端跑通；对接 Agent Control，守住“仅预置算法”边界。

任务分解

- **M1.1 抽取 OpenSAI 客户端 SDK（`opensai_client`）** [已具备基础]：把现有 `inject_collab` / `user_client` 里的“提交 DAG + 等结果”逻辑抽成一个**框架无关**的可复用 Python 模块（封装控制面提交、dagResult 等待、设备/监控查询）。插件与未来其他调用方都复用它，避免重蹈 naive TCP framing / user_beat 路由的坑。**注**：目前提交逻辑已在 `inject_collab` / `run_tests` / `test_full` / `test_bridge` / `user_client` 多处各自重复实现一遍——抽 SDK 时应一并收编这些重复实现，同时作为回归测试的统一客户端基座。
- **M1.2 定义 Agent 工具集（tool schema）** [已具备基础]：以下工具的**数据源均已就绪**，Web 桥接也已以 HTTP 端点暴露（`/api/algorithms`、`/api/devices`、`/api/events`、`/api/dag/upload`），本任务主要是在其上封一层稳定、Agent 友好的工具 schema（直连或经桥接均可）：
  - `list_algorithms()` ——返回已注册算法目录（名称、参数、是否产出/是否需输入），数据来自 `device/implements/Algorithm_*`（桥接 `/api/algorithms` 已可用）。

  - `list_devices()` / `cluster_status()` ——读 Redis `OPENSAI_MONITOR_VIEW`（T1.3 聚合视图：设备、槽位、负载、账本、链路；桥接 `/api/devices` 已可用）。
  - `submit_dag(dag_spec)` ——校验后提交，返回 `job_id`。
  - `get_result(job_id)` / `wait_result(job_id, timeout)` ——轮询/等待 dagResult。
  - `submit_task(algorithm, params)` ——单节点便捷封装（可选）。
- **M1.3 Agent 友好的 DAG 描述 schema**：结构化 JSON（节点：id / algorithm / params / beforeNodes-afterNodes / 可选资源提示 cores • bw • mem / 可选 directAllocationId），与现有 `dagData` 对齐但更易由 Agent 生成。
- **M1.4 “仅预置算法”边界校验**：`submit_dag` 校验 ① 只引用已注册算法 ② 无环 ③ 参数合法；非法（未知算法/任意代码）一律拒绝。**服务端二次校验**（纵深防御），不只靠插件层。**现状**：设备端已对未知/不可用算法做**执行期**容错（回 ERROR 结果、不再挂起、放置预留正常释放）；本任务要补的是更靠前的**提交期拒绝**（在 `submit_dag` 与服务端入口处，未知算法/成环/非法参数直接拒收，不进入派发）。
- **M1.5 长任务的同步/异步处理**：卸载是长耗时操作 → 用 `job_id` + 轮询（`get_result`）或带超时的 `wait_result`，适配 Agent 工具调用的超时模型。
- **M1.6 对接 OpenClaw Agent Control**：把 OpenSAI 工具注册到 Agent Control；按策略放行/拦截（允许哪些工具、频率限制）；“仅预置算法”边界同时由 M1.4 校验与 Agent Control 策略双重保证。
- **M1.7 打包为原生 AgentSkill**：`skills/opensai/` 自包含 Python 模块（清单/元数据、自动发现）、配置（OpenSAI server/Redis 地址）、依赖声明。
- **M1.8 端到端 demo**：Agent 用自然语言描述一个感知/计算任务 → 插件构造预置算法 DAG（如协同感知 FeatureExtract+Fusion，或 ProduceBlob→ConsumeBlob）→ 卸载到设备群 → 返回结果（现有 Windows 栈）。
- **M1.9 测试**：插件单测（schema 校验、非预置算法拒绝）、集成测（插件 → 栈 → 结果）并入回归。

交付物

`opensai_client` SDK；`skills/opensai/` 插件模块 + 工具 schema；Agent DAG 描述规范文档；Agent Control 策略配置；端到端 demo 脚本；插件单测 + 集成测。

依赖

现有 server/device/user_client 栈、Redis；**OpenClaw 运行时 + Agent Control（需确认版本与接口，见 § 开放事项）**。

风险与应对

- OpenClaw 插件契约的细节在拿到运行时前不确定 → **把 OpenClaw 专用胶水隔离在薄适配层，核心逻辑放在框架无关的 `opensai_client`**。
- Agent 工具超时 vs 长任务 → `job_id` + 轮询模式。
- 预置边界绕过 → 服务端二次校验。
- 现有控制面 framing/路由坑 → 一律走抽取后的稳定客户端路径。

验收标准

- Agent 能列算法/设备、提交预置算法 DAG、取回**正确结果**，现有栈端到端跑通。
- 引用未注册算法或试图提交任意代码的 DAG **被拒绝**（边界成立）。
- Agent Control 能拦截一次被禁止的工具调用。
- 回归套件 + 新增插件集成测**全绿**。

M2 · Tailscale 跨网组网

目标：让不同 LAN/NAT 下的设备经一个 Tailscale tailnet 加入同一 OpenSAI 集群——控制面（设备 ↔server TCP）与数据面（设备 ↔ 设备 QUIC）都在 tailnet 上工作——使 M1 的插件能把任务分发到**真正跨网**的设备。各端安装官方 Tailscale 客户端。

任务分解

- **M2.1 tailnet 建立与设备纳管：**开通 tailnet；基于 auth key 的设备接入（ephemeral/tagged key）；各 OS 安装文档（Win/Linux/macOS；Android 客户端供瘦设备用）。
- **M2.2 地址/配置改造：**把硬编码的 `127.0.0.1`/LAN 地址改为 tailnet 地址；server 公布其 tailnet IP / MagicDNS 名；设备注册其 tailnet IP（大脑已跟踪设备 IP/端口用于 D2D，指向 tailnet IP 即可）。新增配置：`OPENSAI_SERVER_ADDR`、设备数据端口经 tailnet 可达。
- **M2.3 服务发现：**server 经 MagicDNS（如 `opensai-server.<tailnet>`）发现，或保留协调器/Redis 注册表但填入 tailnet 地址；D2D QUIC 用注册表中的 tailnet IP 寻址。
- **M2.4 QUIC-over-WireGuard 验证** [已具备基础]：验证 aioquic D2D 经 tailnet（UDP 走 WireGuard）可用；测 MTU/吞吐影响；评估 aioquic ~10MB/s 上限 + WireGuard 开销叠加并调优。**注：**本版**已实现多链路数据面（中继 + 多网卡并行聚合）**，可直接用于缓解单链路带宽上限、并天然适配 DERP 中继路径，跨网时复用即可。
- **M2.5 连通性鲁棒性** [已具备基础]：处理直连 vs DERP 中继回退（无直连路径时 Tailscale 走中继）及其对时延/吞吐的影响——**大脑放置已是链路感知**（按各设备实测链路带宽 `observed_links` 估传输代价选设备，见 `offloading_brain._transfer_seconds`），跨网只需让 `observed_links` 采到 tailnet/DERP 实测速率即自动生效；另处理设备 IP 变化、断线重连、睡眠/唤醒、离线。
- **M2.6 安全/ACL：**tailnet ACL（哪些设备可达 server 控制端口、哪些可 D2D）；设备身份/标签；与 M1 的 Agent 边界打通（Agent 提交的任务只在被授权设备上跑）；OpenSAI 端口仅绑 tailnet 接口。
- **M2.7 测试：**跨网集成测（≥2 台处于不同网络的机器，或模拟）经 tailnet 接入并跑通一个 DAG；验证 D2D 传输经 tailnet。

交付物

各 OS 的 tailnet 接入文档/脚本；地址/配置改造；服务发现机制；ACL 策略；QUIC-over-tailnet 验证报告；跨网集成测。

依赖

M1（用插件演示跨网卸载）；Tailscale 账号/tailnet；大脑的设备地址处理。

风险与应对

- aioquic 纯 Python ~10MB/s 上限 + WireGuard 开销可能成为跨网数据面瓶颈 → 实测刻画；**多链路（中继 + 多网卡聚合）已实现**，可用于缓解；仅**原生/高吞吐 QUIC** 列为后续（本版不做，见非目标）。
- 直连失败走 DERP 中继 → 时延高/吞吐低；**链路感知放置已实现**（W_XFER 代价已纳入实测链路速率），跨网下让其使用 tailnet/DERP 实测速率即可自动优选更快路径。
- NAT/防火墙边缘情况、移动端睡眠掉连。
- 依赖外部 Tailscale → 已确认接受；文档注明自建（Headscale）为可选项，但本版不实现。

验收标准

- 处于 ≥2 个不同网络的设备经 tailnet 加入同一集群；M1 插件提交的 DAG 跨这些设备执行并返回**正确结果**。
- D2D QUIC 经 tailnet 传输成功，吞吐已刻画。
- ACL 能约束谁可达控制面/数据面。

M3 · 跨平台设备端

目标：设备端在 Win/Linux/macOS 作一等计算节点运行，在 Android 作**瘦客户端**（采集/轻任务/结果展示，不跑重模型），经一个平台抽象层统一。

任务分解

- **M3.1 平台差异盘点 + 抽象层**：定义 `PlatformAdapter` 接口统一资源控制——绑核、限内存、进程池模型、BLAS 线程对齐（已有 threadpoolctl 基础）。实现：Windows（Job Object + cpu_affinity）、Linux（RLIMIT_AS + cpu_affinity）、macOS（**无 cpu_affinity → 优雅退回线程数限制**；RLIMIT_AS 有局限）、Android-thin（最小/无）。
- **M3.2 macOS 一等计算端**：处理 cpu_affinity 缺失（已有部分 fallback）、fork-vs-spawn 进程池启动可靠性、全栈在 macOS 跑通、打包（venv 引导 / py 启动器）。
- **M3.3 Linux 一等计算端**：验证（最接近现状）；RLIMIT_AS 内存上限；cpu_affinity；打包。
- **M3.4 Android 瘦客户端**：轻量设备代理——① 经 tailnet 注册到 server；② 做轻任务/数据采集（如取图、轻预处理或仅中继）；③ **不跑重 torch 模型**；④ 展示结果。MVP 可用 **Termux 托管的 device_client "thin 模式"**（能力标志，不加载重算法）；原生 Kotlin 代理列为后续。定义**设备能力档（compute_class: full/thin）**。
- **M3.5 能力感知放置** [已具备基础]：扩展大脑放置（plan/smart），尊重设备能力档——瘦设备只接轻任务/采集，重算法不会被分配到瘦设备。**插入点已明确**：本版已有的**健康预筛**（`_healthy_devices`，从候选设备中先剔除不合格者）即是加 `compute_class(full/thin)` 能力过滤的天然位置，工作量较小。
- **M3.6 各平台打包与安装**：每个桌面 OS 一键引导；Android（Termux）安装文档。
- **M3.7 跨平台 CI/测试矩阵**：回归套件（或子集）在 Win/Linux/macOS 运行；Android 瘦客户端集成测（或模拟）。

交付物

`PlatformAdapter` + 各 OS 实现；macOS/Linux 验证栈 + 打包；Android 瘦客户端 + 能力档；能力感知放置；跨平台测试矩阵。

依赖

M2（Android 瘦客户端经 tailnet 接入；异构多端跨网协作）；M1（最终目的是 Agent 能用异构设备池）。桌面跨平台可先在局域网验证。

风险与应对

- macOS spawn/fork + 无 cpu_affinity → 资源控制弱化（用 fallback；文档说明局限）。
- Android Termux Python 脆弱（后台限制、不保证常驻）→ 瘦范围降低风险；重计算明确排除。
- 各平台 numpy/torch 构建异构。

验收标准

- 设备端在 Windows/Linux/macOS 上运行并通过回归（或平台子集）。
- Android 瘦客户端经 tailnet 注册、收并完成一个轻/采集任务；大脑**绝不**给它分配重模型。
- 混合 OS 集群（如 Win+Linux+macOS+Android-thin）正确执行插件提交的 DAG。

横切关注点

- **统一安全模型（三层）**：tailnet ACL（网络层）+ Agent Control 与“仅预置算法”（Agent 层）+ 设备能力/授权（设备层）；文档化威胁模型与边界。
- **打包/分发**：各 OS 安装/引导；评估弱化 Redis 硬依赖（当前必需）——可列为后续（让 Redis 可选/内嵌），本版不移除。
- **配置管理**：统一配置设备角色/能力档、tailnet 地址、插件设置、运行时旋钮（placement / W_XFER / W_LOAD / AUTO_MEM 等）。
- **可观测性** [已具备基础]：聚合监控视图 `OPENSAI_MONITOR_VIEW` 已含设备资源、账本与**链路信息**（`observed_links`），并已有决策事件流与 Web 桥接 HTTP 端点；本版只需再补**平台 / 能力档**字段，供插件 `cluster_status` 使用。
- **文档与演示**：每个里程碑配套。
- **测试矩阵**：平台 × 网络 × 插件。

架构改动总览

- **新增**：`opensai_client` SDK（框架无关客户端）、`skills/opensai/` 插件、`PlatformAdapter` 抽象层 + 各 OS 实现、Android 瘦客户端代理。
- **改造**：地址/配置 tailnet 化、放置加入设备能力约束、监控视图扩展。
- **复用**：现有 server 大脑（**已链路感知**的智能放置 + 四维资源控制）、device 进程池 / **多链路数据面（中继 + 多网卡）**、Redis 聚合监控视图、**决策事件流 + Web 桥接 HTTP 接口**、**前端控制台**、**结构化回归（测试计划 + 30 条综合功能断言 + 多链路/事件/桥接断言）**。